

Fiche TP N° : 1 (Introduction aux Threads)

Thread : définition et caractéristiques

Un thread est un fil d'exécution de code, à l'intérieur d'un processus, et qui a la possibilité d'être ordonnancé. Il s'agit d'une version allégée d'un processus.

- Tout processus a un thread principal, depuis lequel d'autres threads peuvent être lancés (le cas d'une application multi-thread).
- Processus et threads partagent certaines propriétés, mais ne peuvent en aucun cas être interchangés.
- Les threads d'un même processus partagent le même espace d'adressage. Toutes les ressources du processus peuvent être accédées par tous les threads, ce qui n'est pas le cas entre deux processus distincts.
- Chaque thread possède son propre compteur programme (PC), un ensemble de registres, un état, et une pile. Chaque thread possède sa propre pile.
- Les variables globales sont partagées entre les threads

Le langage **C** n'offre pas de mécanisme natif de gestion des threads. Mais, il existe des bibliothèques spécifiques qui accomplissent ce rôle. Dans nos TPs, nous allons utiliser la bibliothèque Pthread (voir TP1).

Création de threads

```
int pthread_create (pthread_t *thread , pthread_attr_t *attr, void *nomfonction, void *arg );
```

Le service pthread_create() crée un processus léger qui exécute la fonction **nomfonction** avec l'argument **arg** et les attributs **attr**. Les attributs permettent de spécifier la taille de la pile, la priorité, la politique de planification, etc.

Suspension de threads

```
int pthread_join(pthread_t *thid, void **valeur_de_retour);
```

pthread_join() : suspend l'exécution d'un processus léger jusqu'à ce que termine le processus léger avec l'identificateur **th_id**. Il retourne l'état de terminaison du processus léger.

Terminaison de threads

```
void pthread_exit(void *valeur_de_retour);
```

pthread_exit() : permet à un processus léger de terminer son exécution, en retournant l'état de terminaison.

Exemple de création d'un thread fils

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void *function_thread(void *arg){
    printf("Je suis un thread fils \n");
}

int main(){

    pthread_t my_thread;
    printf("In main: Je suis le père \n");
    pthread_create(&my_thread, NULL, &function_thread, NULL);
    system("PAUSE");
    return 0 ;
}
```